

P3358R0

SARIF for Structured Diagnostics

Published Proposal, 2024-07-16

This version:

<https://wg21.link/P3358R0>

Author:

[Sy Brand](#)

Audience:

SG15

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

[Static Analysis Results Interchange Format](#) (SARIF) is a standardised, structured format for the output of static analysis tools. Support for this format is common across C++ tooling, including static analyzers and compilers. Adoption is also growing among editors and IDEs. This paper presents what SARIF is, how it benefits the C++ ecosystem, the current state-of-the-art in support, and a future direction that will enrich the C++ diagnostic experience for programmers.

Table of Contents

- 1 Overview of SARIF**
- 2 What SARIF Gives Us**
- 3 SARIF Adoption in C++ Tools**
 - 3.1 Compilers
 - 3.1.1 MSVC
 - 3.1.2 GCC

3.1.3 Clang

3.2 Static Analyzers

3.3 IDEs/Editors

3.3.1 Others

4 Hierarchical Diagnostics

5 Suggested Direction

5.1 SARIF Standard

5.2 Build Systems

5.3 Compilers

5.4 IDEs

§ 1. Overview of SARIF

SARIF is a [JSON](#)-based format for the output of static analysis tools (and therefore also compilers: what is a compiler if not a static analysis tool that happens to also output code?) It is standardised under the [OASIS Open](#) project. The most recent standard version is [v2.1.0](#). The goals of the project are to:

- Comprehensively capture the range of data produced by commonly used static analysis tools.
- Be a useful format for analysis tools to emit directly, and also an effective interchange format into which the output of any analysis tool can be converted.
- Be suitable for use in a variety of scenarios related to analysis result management and be extensible for use in new scenarios.
- Reduce the cost and complexity of aggregating the results of various analysis tools into common workflows.
- Capture information that is useful for assessing a project's compliance with corporate policy or certification standards.
- Adopt a widely used serialization format that can be parsed by readily available tools.
- Represent analysis results for all kinds of artifacts, including source code and object code.

SARIF diagnostics are captured in [UTF-8](#)-encoded JSON objects with a specific set of JSON properties. Such objects are referred to as [sarifLog](#) objects, and capture the results of one or more analysis runs, potentially from multiple tools. They contain metadata about the analysis runs and nested information about each diagnostic produced by the runs.

Consider the following C++ code as an example:

```
int main() {  
    int oops = "not an int"  
}
```

This code has two errors and one issue that's commonly diagnosed as a warning:

- The type of `oops` is wrong
- There's a missing semicolon
- `oops` is unused in the rest of the program

GCC 14.1 generates the following `sarifLog` object when it compiles the above code with `-Wall -fdiagnostics-format=sarif`:

```
{  
  "$schema": "https://raw.githubusercontent.com/oasis-tcs/sarif-spec/master/  
  "version": "2.1.0",  
  "runs": [  
    {  
      "tool": {  
        "driver": {  
          "name": "GNU C++17",  
          "fullName": "GNU C++17 (Compiler-Explorer-Build-gcc--b",  
          "version": "14.1.0",  
          "informationUri": "https://gcc.gnu.org/gcc-14/",  
          "rules": [  
            {  
              "id": "-fpermissive",  
              "helpUri": "https://gcc.gnu.org/onlinedocs/gcc-  
            },  
            {  
              "id": "-Wunused-variable",  
              "helpUri": "https://gcc.gnu.org/onlinedocs/gcc-  
            }  
          ]  
        },  
        "invocations": [  
          {  
            "executionSuccessful": true,
```

```

    "toolExecutionNotifications": []
  },
  "artifacts": [
    {
      "location": {
        "uri": "<source>"
      },
      "contents": {
        "text": "int main() {\n    int oops = \"not an int\"
      },
      "sourceLanguage": "cplusplus"
    }
  ],
  "results": [
    {
      "ruleId": "-fpermissive",
      "level": "error",
      "message": {
        "text": "invalid conversion from 'const char*' to
      },
      "locations": [
        {
          "physicalLocation": {
            "artifactLocation": {
              "uri": "<source>"
            },
            "region": {
              "startLine": 2,
              "startColumn": 16,
              "endColumn": 28
            },
            "contextRegion": {
              "startLine": 2,
              "snippet": {
                "text": "    int oops = \"not an in
              }
            }
          },
          "logicalLocations": [
            {
              "name": "main",
              "fullyQualifiedName": "main",

```

```

        "decoratedName": "main",
        "kind": "function"
    }
}
]
}
]
},
{
    "ruleId": "error",
    "level": "error",
    "message": {
        "text": "expected ',', or ';' before '}' token"
    },
    "locations": [
        {
            "physicalLocation": {
                "artifactLocation": {
                    "uri": "<source>"
                },
                "region": {
                    "startLine": 3,
                    "startColumn": 1,
                    "endColumn": 2
                },
                "contextRegion": {
                    "startLine": 3,
                    "snippet": {
                        "text": "}\n"
                    }
                }
            },
            "logicalLocations": [
                {
                    "name": "main",
                    "fullyQualifiedName": "main",
                    "decoratedName": "main",
                    "kind": "function"
                }
            ]
        }
    ]
},
{

```

```

    "ruleId": "-Wunused-variable",
    "level": "warning",
    "message": {
      "text": "unused variable 'oops'"
    },
    "locations": [
      {
        "physicalLocation": {
          "artifactLocation": {
            "uri": "<source>"
          },
          "region": {
            "startLine": 2,
            "startColumn": 9,
            "endColumn": 13
          },
          "contextRegion": {
            "startLine": 2,
            "snippet": {
              "text": "    int oops = \"not an in"
            }
          }
        },
        "logicalLocations": [
          {
            "name": "main",
            "fullyQualifiedName": "main",
            "decoratedName": "main",
            "kind": "function"
          }
        ]
      }
    ]
  }
}

```

The output has three top-level properties:

- `$schema`: Defines the [JSON Schema](#) for the `sarifLog` object, which is provided by the SARIF project
- `version`: Defines the SARIF version
- `runs`: Captures the analysis runs that produced diagnostics (in this case, there is only one)

The `runs` property is an array of `run` objects (defined in the SARIF spec), which contains metadata about the tool that produced the diagnostics in the `tool` property, metadata about how the tool was executed in the `invocations` property, the source code that the tool was run on in the `artifacts` property, and the diagnostics produced in the `results` property. Some properties are mandatory, some are optional. There are several optional properties that GCC did not include in this output.

The diagnostics in the `results` property are `result` objects. Each of the results produced by GCC have the following properties:

- `ruleId`: An identifier key that refers to an entry in the `tool.driver.rules` object for this run, which provides more information about what analysis rule caused this diagnostic to be generated.
- `level`: The severity of the diagnostic (`error`, `warning`, `note`, or `none`).
- `message`: The textual diagnostic that should be displayed to the user.
- `locations`: Descriptions of the parts of the source code to which the diagnostic applies.

§ 2. What SARIF Gives Us

In [P2429](#) I presented the state-of-the-art of compiler diagnostics, both in research and in industry tooling. The paper "[Compiler Error Messages Considered Unhelpful: The Landscape of Text-Based Programming Error Message Research](#)" summarized the following key ways in which compiler errors can be improved:

- Increase readability by using plain language, being concise, and writing errors for humans rather than tools.
- Reduce cognitive load by placing relevant information near the offending code, reducing redundancy so the user does not process the same information twice, and using multiple modalities to provide feedback.
- Provide context that can help the user.
- Use a positive tone
- Show examples of similar errors that are minimal and aid understanding.

- Show solutions or hints.
- Allow dynamic interaction by providing the user with autonomy over error message presentation.
- Provide scaffolding that helps the user connect concepts in the language with errors in their code.
- Use logical argumentation by providing a coherent narrative of the error.
- Report errors at the right time by giving the user the right amount of information when they need it.

SARIF supports several of these points. By providing diagnostics in a machine-readable format, tools can more easily filter and manipulate diagnostics in order to reduce cognitive load. Solutions and hints can be provided in a standardized manner with [fix objects](#). Dynamic interaction and logical argumentation can be more easily facilitated by compilers and IDEs because they can express and understand the hierarchical nature of diagnostics (this is how Visual Studio's [Problem Details Window](#) works).

Furthermore, since SARIF is standardised and there are existing tools that can read, manipulate, and visualize it (see [§ 3 SARIF Adoption in C++ Tools](#) for some examples), users can take the output of C++ compilers and use external tools to process them.

§ 3. SARIF Adoption in C++ Tools

§ 3.1. Compilers

§ 3.1.1. MSVC

SARIF support for MSVC is documented on the [Structured SARIF Diagnostics](#) page and is available as of Visual Studio 2022 version 17.8.

There are two ways to make the MSVC compiler produce SARIF diagnostics:

- Pass the `/experimental:log FILENAME` switch on the command line. The `FILENAME` argument specifies where to output SARIF diagnostics. The `.sarif` suffix is added to `FILENAME` to produce the final filename at which to store the resulting SARIF diagnostics. `FILENAME` can be absolute, or relative to the current working directory of the compiler.
- Launch `cl.exe` programmatically and set the `SARIF_OUTPUT_PIPE` environment variable to retrieve SARIF blocks through a pipe.

To retrieve SARIF through a pipe, tools set the `SARIF_OUTPUT_PIPE` environment variable to be the UTF-16-encoded integer representation of the `HANDLE` to the write end of the pipe, then launch `cl.exe`. SARIF is sent along the pipe as follows:

- When a new diagnostic is available, it is written to this pipe.
- Diagnostics are written to the pipe one-at-a-time rather than as an entire SARIF object.
- Each diagnostic is represented by a [JSON-RPC 2.0](#) message of type [Notification](#).
- The JSON-RPC message is prefixed with a [Content-Length](#) header with the form [Content-Length: N](#) followed by two newlines, where [N](#) is the length of the following JSON-RPC message in bytes.
- The JSON-RPC message and header are both encoded in UTF-8.
- This JSON-RPC-with-header format is compatible with [vs-streamjsonrpc](#).
- The method name for the JSON-RPC call is [OnSarifResult](#).
- The call has a single parameter that is encoded [by-name](#) with the parameter name [result](#).
- The value of the argument is a single [result](#) object as specified by the SARIF Version 2.1 standard.

[Content-Length](#): 334

```
{"jsonrpc":"2.0","method":"OnSarifResult","params":{"result":{"ruleId":"C10
```

The SARIF [result](#) object additionally encodes information about hierarchical diagnostics. See [§ 4 Hierarchical Diagnostics](#) for details.

§ 3.1.2. GCC

GCC supports outputting diagnostics in SARIF 2.1 as of GCC 13. It is controlled with the `-fdiagnostics-format=FORMAT` option, where the valid values for `FORMAT` are `sarif-file`, `sarif-stderr`, `json-file`, and `json-stderr`. When `sarif-file` or `json-file` are passed, the resulting SARIF or JSON data is stored in `FILENAME.sarif` in the current working directory of the compiler where `FILENAME` is the filename of the source file whose translation unit.

The file format used for `json-file` and `json-stderr` carries essentially the same information as the SARIF, but in a custom JSON format.

§ 3.1.3. Clang

As of version 15, Clang has "unstable" support for SARIF 2.1 output. It is controlled with the `-fdiagnostics-format=FORMAT` option, where the `FORMAT` argument can be `clang`, `msvc`, `vi`, `sarif`, or `SARIF`. The final two options are undocumented. When invoked with `-fdiagnostics-format=sarif` or `-fdiagnostics-format=SARIF`, Clang outputs SARIF to `stderr`, along with the following message:

```
clang++: warning: diagnostic formatting in SARIF mode is currently unstable
```

There is an [open pull request](#) that changes this support to mirror GCC's `sarif-file` and `sarif-stderr` options.

§ 3.2. Static Analyzers

- [MSVC Code Analysis](#) - Provides results to Visual Studio in SARIF 2.1.
- [Qodana](#) - [Supports SARIF 2.1](#).
- [Clang-Tidy](#) - There is an [open issue](#) for SARIF support.
- [GitHub CodeQL](#) - [Supports SARIF 2.1](#).
- [SonarQube](#) - Can [import SARIF 2.1](#).
- [CodeChecker](#) - [Supports SARIF 2.1](#).
- [Infer](#) - [Supports SARIF 2.1](#)

§ 3.3. IDEs/Editors

- [Visual Studio Code](#) - There is a [SARIF viewer extension](#).
- [Visual Studio](#) - The [Problem Details Window](#) is driven by SARIF. When building a [Visual Studio project](#) (MSBuild) and the **Project > Properties > Advanced > Enable MSVC Structured Output** option is enabled, Visual Studio will spawn `cl.exe` with the `SARIF_OUTPUT_PIPE` environment variable set and will stream SARIF results out of the compiler and into the [Output Window](#) and Problem Details Window. As such, Visual Studio supports SARIF natively when building with MSVC and MSBuild. There is also a [SARIF viewer extension](#).
- [Android Studio](#), [Rider](#), and [CLion](#) - There is a [SARIF viewer extension](#).

- [Compiler Explorer](#): There is an [open issue](#) for SARIF support.

§ 3.3.1. Others

- [CTest](#) - There is an [open issue](#) for SARIF support.
- [Convisio Platform](#) - [Supports SARIF](#)

§ 4. Hierarchical Diagnostics

One key benefit of using a structured diagnostic format is the ability to output diagnostics that have a logical hierarchy. This is especially useful for code that uses Concepts heavily (which includes any piece of code that uses Ranges) Consider, for example, the following C++ code:

```
struct dog {};  
struct cat {};  
  
void pet(dog);  
void pet(cat);  
  
template <class T>  
concept has_member_pet = requires(T t) { t.pet(); };  
  
template <class T>  
concept has_default_pet = T::is_petable;  
  
template <class T>  
concept pettable = has_member_pet<T> or has_default_pet<T>;  
  
void pet(petable auto t);  
  
struct lizard {};  
  
int main() {  
    pet(lizard{});  
}
```

Passing a `lizard` to `pet` is not valid, because neither of the `pet(dog)` or `pet(cat)` functions match, and the template overload requires the type model `pettable`, which `lizard` does not, since

it neither has a member `pet` function nor specifies `lizard::is_petable`. MSVC generates a hierarchical error like this:

```
source.cpp(21,5): error C2665: 'pet': no overloaded function could convert
source.cpp(5,6):
could be 'void pet(cat)'
    source.cpp(21,5):
    'void pet(cat)': cannot convert argument 1 from 'lizard' to 'cat'
    source.cpp(21,15):
    No user-defined-conversion operator available that can perform
source.cpp(4,6):
or      'void pet(dog)'
    source.cpp(21,5):
    'void pet(dog)': cannot convert argument 1 from 'lizard' to 'dog'
    source.cpp(21,15):
    No user-defined-conversion operator available that can perform
source.cpp(16,6):
or      'void pet(_T0)'
    source.cpp(21,5):
    the associated constraints are not satisfied
    source.cpp(16,10):
    the concept 'pettable' evaluated to false
    source.cpp(14,20):
    the concept 'has_member_pet' evaluated to false
    source.cpp(8,44):
    'pet': is not a member of 'lizard'
    source.cpp(18,8):
    see declaration of 'lizard'
    source.cpp(14,41):
    the concept 'has_default_pet' evaluated to false
    source.cpp(11,30):
    'is_petable': is not a member of 'lizard'
    source.cpp(18,8):
    see declaration of 'lizard'
source.cpp(21,5):
while trying to match the argument list '(lizard)'
```

Note that each potential overload of `pet` is considered and reasons for why the candidate is not valid are given for each as nested diagnostics. Furthermore, the constraint failures for `pettable` are further nested, giving the reasons that the constituent constraints failed as well.

This hierarchy is encoded in SARIF like so (heavily excerpted to only have relevant information):

```
{
  "jsonrpc": "2.0",
  "method": "OnSarifResult",
  "params": {
    "result": {
      "message": {
        "text": "'pet': no overloaded function could convert all th
      },
      "locations": [
        //snip
      ],
      "relatedLocations": [
        //snip
        {
          "message": {
            "text": "or      'void pet(_T0)'"
          }
        },
        {
          "message": {
            "text": "the associated constraints are not satisfi
          },
          "properties": {
            "nestingLevel": 1
          }
        },
        {
          "message": {
            "text": "the concept 'pettable<lizard>' evaluated t
          },
          "properties": {
            "nestingLevel": 2
          }
        },
        {
          "message": {
            "text": "the concept 'has_member_pet<lizard>' evalu
          },
          "properties": {
            "nestingLevel": 3
          }
        }
      ]
    }
  }
}
```

```

    },
    {
      "message": {
        "text": "'pet': is not a member of 'lizard'"
      },
      "properties": {
        "nestingLevel": 4
      }
    },
    {
      "message": {
        "text": "see declaration of 'lizard'"
      },
      "properties": {
        "nestingLevel": 4
      }
    },
    {
      "message": {
        "text": "the concept 'has_default_pet<lizard>' evaluated to false"
      },
      "properties": {
        "nestingLevel": 3
      }
    },
    {
      "message": {
        "text": "'is_petttable': is not a member of 'lizard'"
      },
      "properties": {
        "nestingLevel": 4
      }
    },
    {
      "message": {
        "text": "see declaration of 'lizard'"
      },
      "properties": {
        "nestingLevel": 4
      }
    },
    //snip
  ]

```

```
}  
  }  
}
```

The compiler outputs SARIF that may include additional information to represent the nested structure of some diagnostics. A diagnostic may contain a "diagnostic tree" of additional information in its `relatedLocations` field. This tree is encoded using a SARIF [property bag](#) as follows:

A location object's `properties` field may contain a `nestingLevel` property whose value is the depth of this location in the diagnostic tree. If a location doesn't have a `nestingLevel` specified, the `depth` is considered to be 0 and this location is a child of the root diagnostic represented by the `result` object containing it. Otherwise, if the value is greater than the depth of the location immediately preceding this location in the `relatedLocations` field, this location is a child of that location. Otherwise, this location is a sibling of the closest preceding location in the `relatedLocations` field with the same depth.

Property bags in SARIF allow tools to generate SARIF with extended information that some SARIF consumers can use to display additional information. As such, the `properties.nestingLevel` property of a `result` object, while supported by the standard, is not understood by other tools.

Clang is considering adopting hierarchical diagnostics in addition to MSVC, and there's an [RFC for it](#).

§ 5. Suggested Direction

This section captures a direction that I propose tooling and the SARIF standard take in order to make the best experience for C++ users.

§ 5.1. SARIF Standard

The SARIF standard should adopt a standard way for expressing hierarchical diagnostics. There is an [existing issue](#) on the SARIF standard GitHub page that is tracking this.

§ 5.2. Build Systems

IDEs and other tools that interact with build systems need a way to retrieve SARIF from running builds. It would likely be possible for tools to find SARIF files on disk (so long as the relevant com-

mand line flags are passed to the compiler) and read them, but this requires the entire compilation to complete before diagnostics can be shown to users. This could be a problem for compilations that take a long time (for example, ones with huge template instantiation trees, or unity builds).

A more user-friendly approach is to enable streaming SARIF from the compiler to the IDE, facilitated by the build system. For example, the build system could use a similar approach to that currently used by MSVC and MS Build by opening a named pipe in a set location that tools can read from in order to retrieve SARIF `result` objects on the fly.

In addition, compiler-agnostic build tools such as CMake should ideally have a way to enable streaming SARIF output on any compiler that supports it. This would require marshalling the data all the way from the compiler, through the native build system, through CMake, and potentially to an IDE. For example, a user could add something like this to their `CMakeLists.txt` file:

```
target_compile_features(my_target PRIVATE sarif_streaming)
```

This would set up the build in such a way that IDEs can retrieve the streamed data using the specified method.

§ 5.3. Compilers

MSVC, Clang, and GCC all support SARIF in some form. However, all three support producing it in slightly different ways:

- MSVC: anonymous pipe or filesystem
- GCC: `stderr` or filesystem
- Clang: `stderr` only

Furthermore, the command line options for the filesystem output for MSVC and GCC are different: GCC computes a filename based on the input source file, producing one SARIF file per source file, whereas MSVC puts everything into a single SARIF file with a given name.

MSVC is the only compiler that supports the streaming of individual SARIF `result` objects during the compilation in an easily-consumable way. Ideally, all three compilers would support both writing out to a file, and streaming `result` objects. For example, GCC's command line syntax could be extended to support `-fdiagnostics-format=sarif-rpc-stderr`, in which case `result` objects would be streamed out to `stderr` during compilation using the JSON-RPC format that MSVC currently uses.

MSVC is the only compiler that supports hierarchical diagnostics deeper than two levels, using the extension specified in § 4 Hierarchical Diagnostics. GCC and Clang produce two levels of hierarchy with the `location` and `related-locations` properties of `result` objects. Ideally, all compilers would support this, especially for concepts errors. For example, GCC currently has the `-fconcepts-diagnostic-depth` flag that controls how deep to issue diagnostics for Concepts. One could imagine this depth making it into the SARIF object and expressed explicitly in the diagnostic hierarchy.

§ 5.4. IDEs

As noted in § 3.3 IDEs/Editors, some IDEs and editors have native support for SARIF and some have extensions that can visualize SARIF information. Ideally, major C++ IDEs will be able to show hierarchical SARIF information produced by compilers while the compilation is executing, in a way similar to Visual Studio's Problem Details Window.